Astrolabe: A Content-Addressable Hypergraph for Semantic Knowledge Management

Xinze Li* Simone Severini†

April 1, 2026

Abstract

We introduce Astrolabe, a content-addressable hypergraph for semantic knowledge management. An astrolabe store is a finite set of entries, each identified by the SHA-256 hash of its content. Each entry carries an ordered reference list (`ref`) pointing to other entries and an opaque record string. References are unconstrained: entries can reference entries of any degree, yielding a generalized hypergraph structure similar to HyperGraphDB but with content-addressable identity and automatic hash propagation. The store admits two orthogonal decompositions: by degree (reference count) and by stage (reference depth).

We also present LeanNets, a plugin that bridges informal (LaTeX) and formal (Lean 4) mathematics by extracting a directed semantic network from the store, where each edge carries the semantic nature of its dependency.

Astrolabe is implemented as an open-source Tauri desktop application with interactive visualization and an extensible plugin architecture, available at `https://github.com/factorin-dev/Astrolabe`.

Contents

*Department of Mathematics, University of Toronto. `xbryanli.li@mail.utoronto.ca`
†Department of Computer Science, University College London, and Google Cloud

1 Introduction

Knowledge management tools broadly fall into two categories: document-centric systems (wikis, note-taking apps) that store prose but lose structural relationships, and graph databases that capture relationships but require schema design and lack accessible authoring interfaces. A recurring challenge across both categories is representing **semantic** relationships. In a mathematical knowledge base, for instance, knowing **that** Theorem A depends on Definition B is less informative than knowing **how**: does A unfold B, rewrite by B, or apply B to a subterm? Tools such as `leanblueprint` [Mas20] have made significant progress by introducing dependency graphs for formalization projects; Astrolabe builds on this foundation by adding semantic content to each dependency edge.

In this paper, we present Astrolabe, a content-addressable data structure designed for semantic knowledge management. In §2, we define the core data structure and two orthogonal decompositions: by degree (how many references an entry has) and by stage (the depth of the reference chain). In §3, we describe the system architecture, which separates a minimal core from an extensible plugin framework. In §4, we present LeanNets, the first plugin built on this framework, which bridges informal (LaTeX) and formal (Lean 4) mathematics by extracting a directed semantic network from the store.

Acknowledgements

The authors thank Jiaqi Lai and Alejandro Radisic for exploring the early prototype together, without which the data structure presented here would not have taken shape. The authors are grateful to Alex Kontorovich, Leonardo de Moura, and Yevgeny Liokumovich for their encouragement and support, and to the Lean community for valuable feedback.

1.1 Related Work

Note-taking tools such as Obsidian [Obs20], Roam Research [Roa20], and Logseq [Log21] introduced bidirectional links, turning notes into graphs. These links carry no semantic content: `[[page name]]` records a connection but not its nature. Knowledge graphs in the RDF/OWL tradition provide typed edges, but the type vocabulary is a fixed enumeration; adding a new relationship type requires schema modification. Content-addressable systems such as Git and IPFS/IPLD [Pro21] provide immutable, hash-identified objects, but impose fixed object types and do not support higher-dimensional semantic structures.

On the theoretical side, Spivak and Kent [SK12] proposed **ologs** (ontology logs) as a categorical framework for knowledge representation (Figure 1). Objects are types, morphisms are functional relations, and the category-theoretic structure enforces consistency. However, ologs restrict morphisms to functional relations and cannot carry open-ended content. The framework has seen limited software adoption.

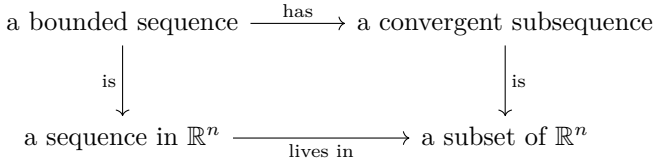
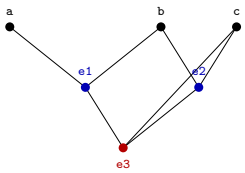


Figure 1: An olog [SK12]: objects are types, morphisms are functional relations, and the diagram commutes.

On the engineering side, HyperGraphDB [Ior10] is a generalized graph database where every entity is an **atom** with a target set (a list of references to other atoms). Atoms with an empty target set are pure nodes; atoms with a non-empty target set are hyperedges. Since hyperedges are themselves atoms, they can be referenced by other hyperedges, yielding higher-order relationships (Figure 2).

1.2 Contributions

Astrolabe adopts the same core abstraction as HyperGraphDB (every entry has an ordered reference list pointing to other entries) but adds two properties:




```
{
  "a":{"target":[]},
  "b":{"target":[]},
  "c":{"target":[]},
  "e1":{"target":["a","b"]},
  "e2":{"target":["b","c"]},
  "e3":{"target":["e1","e2","c"]}
}
```

Figure 2: HyperGraphDB data model: every entity is an atom with a target set. Black = atoms (target = []), blue = hyperedges, red = higher-order hyperedge (e_3 references e_1 and e_2).

1. **Content-addressable identity**: entries are identified by the SHA-256 hash of their content (as in Git/IPFS [Pro21]), providing deduplication, tamper evidence, and decentralized identity.
2. **Automatic hash propagation**: when an entry is modified, its hash changes, and all referencing entries are recursively updated, maintaining referential integrity across the entire network.

The data structure is intentionally **maximally relaxed** to accommodate diverse and complex semantics. The record field is an opaque string; all domain-specific conventions are delegated to plugins, keeping the core format domain-agnostic.

2 Data Model

The core data structure of Astrolabe is a content-addressable store of **entries**, where each entry is a triple of hash, ref, and record. Entries can have arbitrary-length references, forming higher-dimensional semantic structures. The store admits two orthogonal decompositions: by **degree** (how many references) and by **stage** (the depth of the reference chain). We describe each in turn.

2.1 Entry Structure

An **astrolabe store** is a finite set of entries. Each entry consists of an ordered reference list pointing to other entries and an opaque record string, with its identity determined by the hash of its content.

Definition 2.1 (Content-addressable hypergraph)**.** *Let H be a hash function and let Σ^* denote the set of all finite strings over an alphabet Σ. A **content-addressable hypergraph** is a finite set S of entries, where each entry $e \in S$ consists of:*

- *an ordered reference list* $\mathrm{ref}(e) = (r_1, \ldots, r_k)$ *with* $r_i \in S$*, and*
- *a record string* $\mathrm{rec}(e) \in \Sigma^*$*,*

with identity $\mathrm{id}(e) = H(\mathrm{id}(r_1) \| \texttt{0x00} \| \cdots \| \mathrm{id}(r_k) \| \mathrm{rec}(e))$*. The **degree** of e is* $|\mathrm{ref}(e)| - 1$*. An entry of degree 0 is called an **atom**.*

The format is general-purpose and can serve any knowledge management scenario by defining appropriate record conventions. In the current implementation, the store is serialized as a single JSON file (`astrolabe.json`):

```
{
  "<12-char-hash>":{"ref":["<hash>", ...], "record":"<string>"}
}
```

Concretely, each entry has three parts:

- **ref** — an ordered list of hashes, any length. |`ref`| defines the **degree**:
  - degree 0: `ref = [self_hash]` — an atom (base unit)
  - degree 1: `ref = [A, B]` — a binary relation
  - degree k: `ref =` [h_0, h_1, ..., h_k] — a higher-dimensional semantic relation

- **record** — a plain string. The core layer does not interpret it; plugins define conventions for structured content (JSON with `sort`, `source`, `title`, `notes`, etc.).
- **hash** — $\texttt{SHA256}(ref_1 \| \texttt{0x00} \| ref_2 \| \cdots \| record)\texttt{[:12 hex]}$, content-addressable.

Hash propagation. Because entries are content-addressed, modifying a record changes its hash. When this happens, every entry that references the old hash must be updated and re-hashed recursively. Figure 3 illustrates: editing the record of atom `a` changes its hash from `a` to `a′`; the edge `e1` (which references `a`) is updated to reference `a′` and re-hashed to `e1′`; then `e3` (which references `e1`) is updated and re-hashed to `e3′`.

Figure 3: Hash propagation: editing atom `a` changes its hash to `a′` (red). Entry `e1` references `a`, so it is updated and re-hashed to `e1′`. Entry `e3` references `e1`, so it becomes `e3′`. Entries `b`, `c`, and `e2` are unaffected. The implementation uses a visited set to prevent infinite loops in cyclic references; cyclic entries are assigned stage $= -1$ and excluded from structural analysis.

Definition 2.1 does not prevent several pathological cases. Consider an existing edge e_1:

```
"e1":{"ref":["A","B"],"record":"A depends on B"}
```

One could create an entry e_2 with $\text{ref}(e_2) = (\text{id}(e_1))$:

```
"e2":{"ref":["e1"],"record":"some annotation"}
```

Such an e_2 would have degree 0 but would not be a self-contained unit of knowledge; it would be a disconnected wrapper around e_1. The correct way to enrich the semantics of e_1 is to modify $\text{rec}(e_1)$, which changes $\text{id}(e_1)$ and triggers hash propagation (Figure 3) to all entries that reference e_1. Other pathological cases include dangling references (pointing to entries not in S), empty reference lists (yielding degree -1), and duplicate elements in a reference list (which would count the same dependency twice). We exclude all such cases with the following definition.

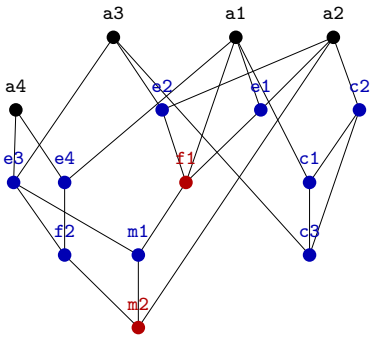
Definition 2.2 (Well-formed content-addressable hypergraph)**.** *A content-addressable hypergraph S is **well-formed** if:*

1. ***(Atom self-reference)*** *Every atom $e \in S$ satisfies $\text{ref}(e) = (\text{id}(e))$.*
2. ***(Identity uniqueness)*** *The map $e \mapsto \text{id}(e)$ is injective on S; that is, distinct entries have distinct hashes.*
3. ***(Referential closure)*** *For every $e \in S$ and every $r_i \in \text{ref}(e)$, we have $r_i \in S$.*
4. ***(Non-empty ref)*** *For every $e \in S$, $|\text{ref}(e)| \geq 1$.*
5. ***(Distinct refs)*** *For every $e \in S$ with $|\text{ref}(e)| > 1$, the elements of $\text{ref}(e)$ are pairwise distinct.*

Throughout this paper, all content-addressable hypergraphs are assumed to be well-formed. The implementation enforces these conditions at entry creation time.

2.2 Degree Decomposition

Figure 4 shows a 16-entry astrolabe file and its reference network. The entries span multiple degrees: 4 atoms (`a1`–`a4`), 4 degree-1 edges (`e1`–`e4`), and higher-degree entries that freely mix atoms and edges in their `ref` lists. The entries `c1`, `c2`, `c3` form a reference cycle. The entry `m2` references `m1` (which itself references `f1`, a degree-2 entry), demonstrating that references are unconstrained: any entry can reference any other entry, regardless of degree.




```
{
  "a1":{"ref":["a1"]},
  "a2":{"ref":["a2"]},
  "a3":{"ref":["a3"]},
  "a4":{"ref":["a4"]},
  "e1":{"ref":["a1","a2"]},
  "e2":{"ref":["a2","a3"]},
  "e3":{"ref":["a3","a4"]},
  "e4":{"ref":["a4","a1"]},
  "f1":{"ref":["a1","e1","e2"]},
  "f2":{"ref":["e3","e4"]},
  "m1":{"ref":["f1","e3"]},
  "m2":{"ref":["m1","f2","a2"]},
  "c1":{"ref":["c2","a1"]},
  "c2":{"ref":["c3","a2"]},
  "c3":{"ref":["c1","a3"]}
}
```

Figure 4: Reference network (left) and its `astrolabe.json` source (right), colored by degree: black = degree 0 (atoms), blue = degree 1, red = degree 2. Entries `f1` and `m2` mix atoms and edges in their `ref`.

2.3 Stage Decomposition

Degree measures how many references an entry has, but not **what those references point to**. Two degree-1 entries can have very different structure: `e1` references two atoms, while `f2` references two edges. The **stage** decomposition captures this depth by classifying entries according to the depth of their reference chains.

Atoms form the base layer. An entry whose refs are all atoms sits one level above; an entry whose refs are all drawn from that level sits one level higher, and so on. We formalize this as follows.

Definition 2.3 (Stage decomposition)**.** *Given a well-formed content-addressable hypergraph S, define the **stage filtration** $F_0 \subseteq F_1 \subseteq \cdots \subseteq S$ inductively:*

- $F_0 = \{e \in S \mid \deg(e) = 0\}$ *(atoms),*
- $F_{m+1} = \{e \in S \mid \forall\, r \in \mathrm{ref}(e),\ r \in F_m\}$.

*The **stage** of an entry e is* $\mathrm{stage}(e) = \min\{m \mid e \in F_m\}$. *In particular, atoms have stage 0, entries whose refs are all atoms have stage 1, and so on. Entries involved in reference cycles belong to no finite stage and are assigned* $\mathrm{stage}(e) = -1$.

Definition 2.4 (Reference cycle)**.** *A **reference cycle** in a content-addressable hypergraph S is a sequence of distinct non-atom entries $e_1, e_2, \ldots, e_n \in S$ $(n \geq 2)$ such that $e_{i+1} \in \mathrm{ref}(e_i)$ for $1 \leq i < n$ and $e_1 \in \mathrm{ref}(e_n)$.*

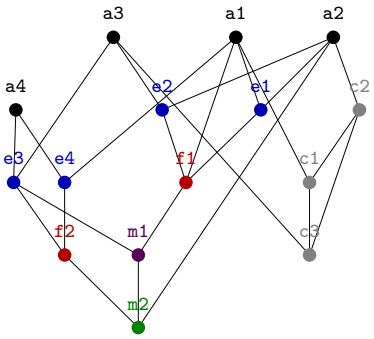
Reference cycles arise naturally in knowledge management. For example, two logically equivalent theorems may each appear in the proof of the other: an edge "A proves B" references atom B, while an edge "B proves A" references atom A, and a higher-degree entry grouping these two edges into an equivalence relation creates a cycle. Figure 5 shows the same network colored by stage instead of degree, where entries `c1`, `c2`, `c3` form such a cycle and are assigned stage $= -1$. The key contrast: `e1` and `f2` both have degree 1, but `e1` is stage 1 (refs are atoms) while `f2` is stage 2 (refs include stage-1 entries).

Proposition 2.5. *For any finite well-formed content-addressable hypergraph S, the stage filtration stabilizes: there exists N such that $F_N = F_{N+1} = \cdots$. The set $S \setminus \bigcup_{m \geq 0} F_m$ is exactly the set of entries involved in reference cycles.*

Proof. Since S is finite and $F_0 \subseteq F_1 \subseteq \cdots \subseteq S$, the chain must stabilize at some F_N.

($\supseteq$) Suppose e belongs to a reference cycle $e_1, \ldots, e_n$. Assume for contradiction that each e_i has finite stage s_i. Since $e_{i+1} \in \mathrm{ref}(e_i)$ and $e_i \in F_{s_i}$, all references of e_i lie in $F_{s_i - 1}$, so $\mathrm{stage}(e_{i+1}) < \mathrm{stage}(e_i)$. Following the cycle yields $\mathrm{stage}(e_1) > \mathrm{stage}(e_2) > \cdots > \mathrm{stage}(e_n) > \mathrm{stage}(e_1)$, a contradiction.

($\subseteq$) If $e \notin \bigcup_m F_m$, then e has some reference $r_1 \notin \bigcup_m F_m$ (otherwise e would belong to some F_m). Repeating, r_1 has a reference $r_2 \notin \bigcup_m F_m$, and so on. Since S is finite, the sequence $e, r_1, r_2, \ldots$ must eventually revisit an entry, forming a reference cycle. □




```
{
  "a1":{"ref":["a1"]},
  "a2":{"ref":["a2"]},
  "a3":{"ref":["a3"]},
  "a4":{"ref":["a4"]},
  "e1":{"ref":["a1","a2"]},
  "e2":{"ref":["a2","a3"]},
  "e3":{"ref":["a3","a4"]},
  "e4":{"ref":["a4","a1"]},
  "f1":{"ref":["a1","e1","e2"]},
  "f2":{"ref":["e3","e4"]},
  "m1":{"ref":["f1","e3"]},
  "m2":{"ref":["m1","f2","a2"]},
  "c1":{"ref":["c2","a1"]},
  "c2":{"ref":["c3","a2"]},
  "c3":{"ref":["c1","a3"]}
}
```


Figure 5: Stage coloring of the same network: black = stage 0 (atoms), blue = stage 1, red = stage 2, purple = stage 3, green = stage 4, gray = cycle (no finite stage).

Reference cycles pose a specific challenge for content-addressable systems that mutable-ID databases such as HyperGraphDB [Ior10] do not face. In HyperGraphDB, atoms are identified by database-generated handles; modifying an atom does not change its identity or affect other atoms. In Astrolabe, identity is derived from content: when the record of an entry e changes, $\mathrm{id}(e)$ changes, and every entry referencing e must be updated and re-hashed recursively. If entries form a reference cycle, this propagation could loop indefinitely.

Astrolabe handles this in two ways. First, hash propagation maintains a visited set: when an entry is encountered a second time during propagation, it is skipped, guaranteeing termination. Second, cyclic entries are assigned $\mathrm{stage}(e) = -1$ and excluded from stage-based structural analysis (degree decomposition and plugin computations proceed normally). This approach allows cycles to exist in the store as valid semantic structures while confining their effects on the rest of the system.

3 Architecture and Extensibility

Based on the astrolabe store defined in §2, we developed a Tauri desktop application for reading, visualizing, and interacting with the data structure. We use a Python backend (FastAPI) to implement the core data model and a React frontend (Next.js) for visualization and authoring; they communicate via REST. All data flows through the astrolabe store as the single source of truth.

3.1 Core Layer

The core is deliberately minimal. It implements entry CRUD, hash computation, and automatic hash propagation (§2), exposed via a REST API. The `record` field is treated as an opaque string: the core knows nothing about "definitions," "theorems," or "proofs." All domain-specific logic is delegated to plugins.

The frontend core provides three workspace panels (Read View, Network View, Detail View), an AI chat assistant via Tauri IPC, and a preprocessor that converts LaTeX-style entry macros (`\entryblock`, `\entryref`) into interactive components (§4.6).

3.2 Plugin Architecture

Each plugin implements the `AstrolabePlugin` interface and is registered in a runtime-toggleable store. A plugin can contribute backend API endpoints (under `/api/plugins/`), frontend UI panels, and record conventions, all without modifying the core. Plugins are typed into three categories:

- **Source**: imports external data into the store. Examples: LaTeX parser, BibTeX importer, Lean 4 artifact parser.
- **Endo**: transforms the store in place. Examples: network metric computation, LLM embedding, metadata enrichment.

- **Sink**: reads the store and produces output. Examples: visualization, export, statistical summary.

A typical workflow composes plugins as Source → Endo → ⋯ → Endo → Sink, with the astrolabe store as the single intermediate representation at every stage. No format conversion is needed between plugins.

3.3 Desktop Shell

Tauri wraps the web frontend into a native desktop application using the system webview, resulting in small binaries and low memory usage. The Python backend is bundled as a sidecar process managed by the Tauri runtime.

4 The LeanNets Plugin

As described in §3, the core is domain-agnostic and all domain logic lives in plugins. In this section, we present **LeanNets**, the first plugin, which turns Astrolabe into a bridge between informal and formal mathematics.

4.1 Motivation

Mathematical knowledge is rich in semantic relationships. A theorem may **unfold** a definition, **rewrite by** a lemma, **apply** a proposition to a subterm, or **specialize** a general result to a particular case. These distinctions matter: knowing **that** Theorem A depends on Definition B is less informative than knowing **how**. Does A unfold B, pattern-match on B's constructors, or merely pass B to another lemma? The `leanblueprint` tool [Mas20] pioneered the use of visual dependency graphs for Lean formalization projects and has become essential infrastructure for large-scale efforts. LeanArchitect [ZMAW26] further automates blueprint generation. On the compiler side, Lean produces fine-grained declaration-level dependency graphs [LPS26]. These tools represent dependencies as directed edges labeled "uses." LeanNets builds on this foundation by adding semantic content to each edge, recording **how** a dependency is used.

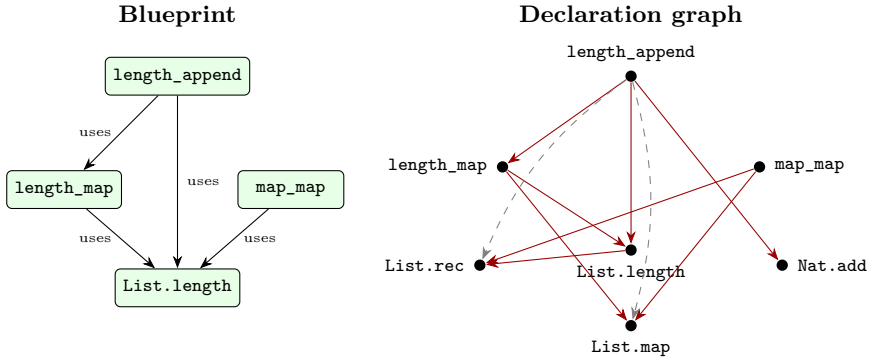


Figure 6: Left: a `leanblueprint` dependency graph. Each edge is labeled "uses" with no further content. Right: a declaration-level dependency graph extracted from Lean 4 compilation artifacts. Both representations lose the **semantic content** of each edge.

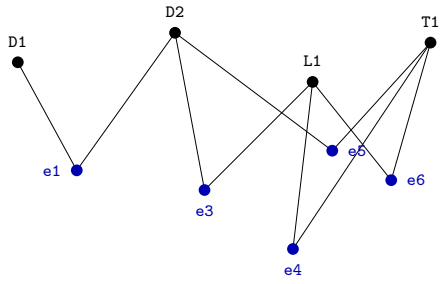
This gap has practical consequences for autoformalization. Large formalization projects are typically organized as blueprint dependency graphs, and autoformalization agents decompose their work along this structure. Harmonic's Aristotle [Har25] uses blueprint-structured decomposition to break problems into lemmas and conjectures; Axiom [Axi25] treats autoformalization as hierarchical planning with dependency retrieval; Math Inc.'s Gauss [Mat26] completed the Strong Prime Number Theorem formalization by working through a `leanblueprint` dependency graph node by node. In all three cases, the dependency annotations available to the agent are limited to "uses," with no distinction between proof strategies.

Astrolabe's data model offers a finer granularity: every edge is an entry with a full `record`, so the nature of each dependency can be made explicit (unfolding, rewriting, specialization, application). Whether this

additional structure improves autoformalization performance is an empirical question we leave to future work.

4.2 Network Extraction

The LeanNets plugin restricts attention to entries in F_1 with deg ≤ 1, that is, atoms and degree-1 entries whose refs are all atoms. This two-dimensional constraint (stage ≤ 1, degree ≤ 1) yields a directed graph: atoms become **nodes**, and degree-1 entries become **directed edges** (`ref[0]` $\rightarrow$ `ref[1]`), with edge semantics derived from the record's `sort` field. Figure 7 shows the entry view; Figure 8 shows the extracted network.




```
{
  "D1":{"ref":["D1"],"record":"definition ..."},
  "D2":{"ref":["D2"],"record":"definition ..."},
  "L1":{"ref":["L1"],"record":"lemma ..."},
  "T1":{"ref":["T1"],"record":"theorem ..."},
  "e1":{"ref":["D2","D1"],
      "record":"D2 uses D1"},
  "e3":{"ref":["D2","L1"],
      "record":"L1 depends on D2"},
  "e4":{"ref":["L1","T1"],
      "record":"T1 cites L1"},
  "e5":{"ref":["D2","T1"],
      "record":"T1 unfolds D2"},
  "e6":{"ref":["T1","L1"],
      "record":"T1 applies L1"}
}
```


Figure 7: Entry view: black = atoms (D1, D2, L1, T1), blue = degree-1 entries (e1–e5) carrying open-ended semantic records.

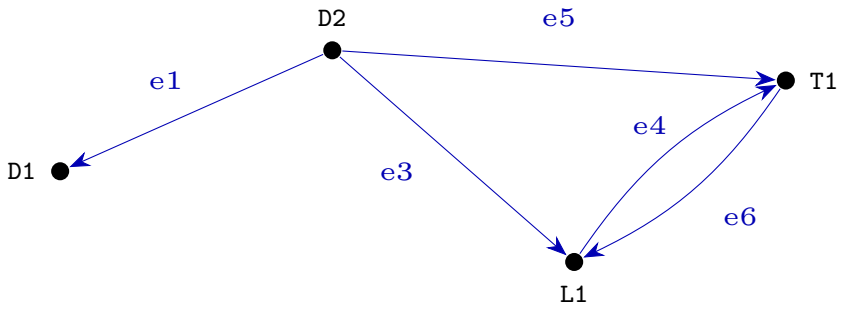


Figure 8: Network view of the same knowledge base. Atoms become nodes; degree-1 entries become directed edges (`ref[0]` $\rightarrow$ `ref[1]`). Edge labels (e1–e5) correspond to the entries in Figure 7.

4.3 Record Conventions

The LeanNets plugin interprets the `record` field as structured JSON with domain-specific fields. Two conventions are currently defined, corresponding to informal and formal mathematics.

Informal mathematics: `source: "tex"`. For informal mathematical content, the `sort` field is derived from the LaTeX environment (e.g., `\begin{theorem}` $\rightarrow$ `sort: "theorem"`). The record carries the mathematical statement in natural language:


```
{"sort":"definition", "source":"tex", "title":"Compact Space",
  "notes":"A subset $K$ is compact if every open cover has a finite subcover."}

{"sort":"theorem", "source":"tex", "title":"Heine-Borel",
  "notes":"A subset of $\mathbb{R}^n$ is compact iff it is closed and bounded."}
```

Atoms are definitions, theorems, lemmas, propositions, corollaries, and examples. Higher-degree entries represent relationships between them: a proof references the theorem it proves and the lemmas it uses. The `notes` field supports LaTeX math and inline references via `\entryref{hash}{text}`.

Formal mathematics: `source: "lean"`. For formal mathematics, the `sort` field mirrors the Lean keyword (`def` → `definition`, `theorem` → `theorem`). The `content` field stores the source code, and `state` tracks formalization progress:

```
{"sort":"theorem", "source":"lean", "title":"List.length_append",
  "state":"proven", "content":"theorem length_append (l1 l2 :  List a) :  ..."}
{"sort":"definition", "source":"lean", "title":"List.length",
  "state":"checked", "content":"def length :  List a -> Nat | [] => 0 | ..."}
```

Formal dependencies can be generated automatically by a source plugin that parses Lean's compilation artifacts. Building on the dependency structure provided by `leanblueprint` [Mas20] and declaration graphs [LPS26], Astrolabe edges carry open-ended content: a dependency can record "rewrites by," "unfolds definition of," or "applies to subterm."

Statement–proof separation. In both conventions, a theorem's **statement** and its **proof** are stored as separate atoms, connected by a degree-1 edge. In Lean, a single `theorem` declaration contains both the type signature (statement) and the tactic body (proof); we split them into two entries during parsing. This separation is motivated by three observations:

- A statement and its proof have different lifecycles: the statement may be stable while the proof is still being revised. Separating them makes hash propagation more precise: changing a proof does not re-hash the statement.
- A single statement may admit multiple proofs (e.g., a constructive proof and a classical proof). Separate atoms represent this naturally.
- The separation mirrors the structure of informal mathematics, where `\begin{theorem}` and `\begin{proof}` are distinct LaTeX environments. This alignment makes cross-source correspondence between `tex` and `lean` entries straightforward: the informal statement maps to the formal statement, and the informal proof sketch maps to the formal proof.

Example (`tex`):

```
"T":{"ref":["T"],
  "record":"{"sort":"theorem","source":"tex","title":"Heine-Borel",
    "notes":"A subset of ℝ^n is compact iff closed and bounded."}"}
"P":{"ref":["P"],
  "record":"{"sort":"proof","source":"tex",
    "notes":"By contradiction, extract a sequence ..."}"}
"e":{"ref":["T","P"],"record":"statement-proof link"}
```

Example (`lean`):

```
"T'":{"ref":["T'"],
  "record":"{"sort":"theorem","source":"lean","title":"IsCompact.isClosed",
    "state":"proven","content":"theorem IsCompact.isClosed (h :  IsCompact s) :  IsClosed s"}"}
"P'":{"ref":["P'"],
  "record":"{"sort":"proof","source":"lean",
    "content":"by intro x hx; exact h.isSeqCompact.isClosed hx"}"}
"e'":{"ref":["T'","P'"],"record":"statement-proof link"}
```

4.4 Cross-Source Edges

Edges (degree-1 entries) connect two atoms. Their record fields fall into two categories:

- **Structural fields** (`sort`, `source`) are **inherited**: they are fully determined by the corresponding fields of the referenced atoms. An edge connecting a `theorem` to a `definition` inherits the sort pair `(theorem, definition)`; an edge between a `tex` atom and a `lean` atom inherits the source pair `(tex, lean)`. No manual annotation is needed for these fields. This inheritance extends naturally to higher-degree entries, whose structural fields are determined by the tuple of their refs' fields.

- **Semantic fields** (`notes`, `title`) are **authored**: they describe the nature of the relationship (e.g., "unfolds definition," "rewrites by lemma") and are the edge's own contribution. This is the layer of information that Astrolabe adds on top of the dependency structure that blueprint tools provide.

The sort pair is auto-derived from the pair of source sorts:

Edge sort pair	Meaning
`(theorem, proof)`	Statement–proof link
`(theorem, definition)`	Depends on definition
`(proof, lemma)`	Proof cites lemma
`(theorem, theorem)`	Cross-source correspondence

When both `tex` and `lean` entries coexist in the same file, a `(theorem, theorem)` edge with one `tex` source and one `lean` source marks the formalization correspondence. This makes the bridge between informal and formal mathematics explicit and navigable in the network view.

4.5 Semantic Propagation

The entry-level hash propagation (§2) maintains data integrity: when an atom changes, all degree-1 entries referencing it are re-hashed. But this does not capture **semantic** dependency. Consider: Definition D changes; Edge E ("Theorem T depends on D") is re-hashed because its ref contains D; but Theorem T itself is unaffected, since its ref is [self] and its record may not mention D's hash. To address this, the LeanNets plugin adds a second layer of propagation on the skeleton graph: when an atom changes, the directed edge structure is traversed in reverse to identify all atoms that **semantically** depend on the changed atom. If Definition D is modified, every atom reachable by following edges backward from D is flagged as affected. This is a straightforward reverse BFS on the skeleton graph. Figure 9 illustrates the difference.

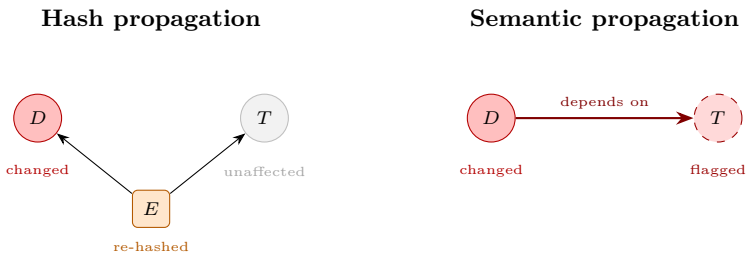


Figure 9: Left: hash propagation re-hashes Edge E (which references D) but does not reach Theorem T, whose `ref` is [self]. Right: semantic propagation on the skeleton graph traverses the dependency edge in reverse, flagging T as affected by the change in D.

4.6 Visualization and Interaction

The network view renders the extracted graph on a 2D HTML Canvas using d3-force. Each node's visual properties encode structural information computed by the backend (NetworkX), with all metrics computed independently per source to prevent cross-source edges from dominating centrality scores. Three aspects of the visual encoding are configurable: size, color, and clustering.

Node **size** is determined by one of the metrics in Table 1, normalized to a visual radius range. Node **color** follows one of the modes in Table 2; the chosen colors propagate to all UI components (network nodes, entry blocks, entry links, detail panel). Nodes can be grouped by **clustering** (Table 3), with a "tightness" slider controlling the intra-cluster attractive force $\mathbf{f}_v = s \cdot \alpha \cdot (\mathbf{c}_{k(v)} - \mathbf{x}_v)$ where $s \in [0, 1]$.

Table 1: Node size metrics.

Metric	Description
uniform	$r_v = c$ (constant)
degree / in-degree / out-degree	$r_v \propto \deg(v)$
PageRank	$\pi = \alpha M\pi + \frac{1-\alpha}{n}\mathbf{1}$
betweenness	$c_B(v) = \sum_{s\neq v\neq t} \sigma_{st}(v)/\sigma_{st}$
Katz	$c_K(v) = \sum_{k=1}^{\infty} \alpha^k (A^k\mathbf{1})_v$
HITS hub / authority	$\mathbf{h} = A\mathbf{a}$, $\mathbf{a} = A^\top\mathbf{h}$
depth	longest path from any root to v
reachability	$\lvert\{u : v \rightsquigarrow u\}\rvert$

Table 2: Node color modes.

Mode	Method
sort	hue = hash(sort) mod 360
community	greedy modularity maximization
layer	gradient by DAG depth
PageRank / betweenness	gradient by metric value
spectral	communities from eigenvectors of $L = D - A$

Table 3: Clustering methods.

Method	Algorithm
Louvain	greedy modularity
sort	partition by `sort` field
stage	partition by DAG depth
spectral	k-means on eigenvectors of L, k from eigengap

Users interact with the graph by clicking nodes (to inspect in the Detail View), dragging (to reposition), scrolling (to zoom), and panning. Selection propagates across all panels. The Read View can embed astrolabe entries inline in `.mdx` documentation via two LaTeX-style macros: `\entryref{hash}{text}` renders an inline clickable link, and `\entryblock{hash}` renders a block-level entry display with sort label, title, and colored left border. Blocks support collapsing (`\entryblock{hash}{collapsible}`) and nesting (`\entryblock{hash}{\entryblock{child}{collapsible}}`). A preprocessor with recursive brace matching converts these macros to React components that fetch entry data and render with sort-derived colors, so documentation stays synchronized with the knowledge network.

5 Future Work

Astrolabe is at an early stage. The data structure and the plugin framework are in place, and LeanNets demonstrates that the approach works for one domain. Much remains to be done. We outline several directions that we plan to explore.

- **AI agent integration.** Exposing the astrolabe store as a set of AI-accessible skills so that users can connect external agents (e.g., Claude Code) directly to the application, enabling structured data operations such as entry creation, reorganization, and semantic analysis through natural-language interaction.
- **Mathematical foundations.** The fragment F_1 with $\deg \leq 1$ (the same fragment used by LeanNets) admits a natural interpretation as a simplicial complex, and tools from algebraic topology (homology,

Betti numbers) can characterize its structural properties. For higher stages and degrees, where entries reference other entries across multiple levels, a unified mathematical theory remains an open question.

- **Higher-stage and higher-degree plugins.** The current LeanNets plugin operates on entries in F_1 with $\deg \leq 1$. Plugins that interpret and visualize entries beyond this fragment (higher stages, ternary relations, multi-way dependencies) would unlock the full expressive power of the data structure.
- **Domain-specific plugins.** The astrolabe store is domain-agnostic. Plugins for other disciplines (software architecture, legal analysis, biological pathways, literature review) would demonstrate the generality of the framework and are a natural next step.
- **Scalable storage.** The current single-JSON-file format is convenient for small to medium knowledge bases but will require a database backend (e.g., SQLite, LMDB) as data grows.

References

[Axi25] Axiom. AXLE: Axiom lean engine, 2025. https://axiommath.ai.

[Har25] Harmonic Team. Aristotle: IMO-level automated theorem proving. *arXiv preprint arXiv:2510.01346*, 2025.

[Ior10] Borislav Iordanov. Hypergraphdb: A generalized graph database. In *Web-Age Information Management (WAIM 2010 Workshops)*, volume 6185 of *LNCS*, pages 25–36. Springer, 2010.

[Log21] Logseq. Logseq — a privacy-first, open-source knowledge base, 2021.

[LPS26] Xinze Li, Nanyun Peng, and Simone Severini. The network structure of Mathlib: Software engineering vs. mathematical dependencies. In preparation, 2026.

[Mas20] Patrick Massot. leanblueprint: A blueprint for lean formalization projects, 2020. https://github.com/leanprover-community/leanblueprint.

[Mat26] Math Inc. Gauss: An agent for autoformalization, 2026. https://www.math.inc/gauss.

[Obs20] Obsidian. Obsidian — a knowledge base that works on local markdown files, 2020.

[Pro21] Protocol Labs. Ipld — interplanetary linked data, 2021.

[Roa20] Roam Research. Roam research — a note-taking tool for networked thought, 2020.

[SK12] David I. Spivak and Robert E. Kent. Ologs: A categorical framework for knowledge representation. *PLoS ONE*, 7(1):e24274, 2012.

[ZMAW26] Thomas Zhu, Pietro Monticone, Jeremy Avigad, and Sean Welleck. LeanArchitect: Automating blueprint generation for humans and AI. *arXiv preprint arXiv:2601.22554*, 2026.